

Evaluating Post-Quantum Cryptographic Readiness on Classical Systems: An End-to-End ML-KEM Implementation in .NET 9 and Angular

Erand Kurtalqi
Faculty of Electrical and Computer
Engineering
University of Prishtina
Prishtina, Kosovo
erand.kurtalqi@student.uni-pr.edu

Mërgim H. Hoti
Faculty of Electrical and Computer
Engineering
University of Prishtina
Prishtina, Kosovo
mergim.hoti@uni-pr.edu

Blerim Rexha *
Faculty of Electrical and Computer
Engineering
University of Prishtina
Prishtina, Kosovo
blerim.rexha@uni-pr.edu

Abstract—The advent of large-scale quantum computers threatens the security of classical public-key cryptosystems such as RSA and elliptic-curve cryptography, whose hardness assumptions are broken by Shor's algorithm. In response, the National Institute of Standards and Technology (NIST) launched a post-quantum standardization process in 2016, culminating in 2024 with the publication of FIPS 203, which specifies the Module-Lattice-Based Key-Encapsulation Mechanism (ML-KEM). While the theoretical security of ML-KEM and its underlying Module Learning With Errors (Module-LWE) problem have been extensively analyzed, its practical readiness within modern full-stack web applications deployed on classical hardware remains under-explored. This paper presents an end-to-end implementation of ML-KEM in a service-oriented architecture built on a .NET 9 backend API and an Angular frontend, leveraging the Bouncy Castle cryptographic library. Performance is empirically evaluated through 1,000 backend iterations and 500 end-to-end HTTPS requests for each parameter set (ML-KEM-512, ML-KEM-768, ML-KEM-1024). Backend key generation, encapsulation, and decapsulation execute in sub-millisecond time, while mean end-to-end latency remains below 3 ms, with cryptographic computation accounting for less than 3% of total request time. Future work will address concurrent workloads, hybrid key exchange, and distributed deployments.

Keywords—post-quantum cryptography, ML-KEM, quantum readiness, classical systems, .NET, Angular, full-stack security, performance evaluation

I. INTRODUCTION

The advancement of quantum computing represents a fundamental challenge to classical public key cryptographic systems that currently secure digital communication, data storage, and networked applications [1], [2]. Algorithms such as RSA and elliptic curve cryptography rely on mathematical problems that are considered computationally infeasible for classical computers but are expected to become vulnerable in the presence of sufficiently powerful quantum adversaries [3]. Classical cryptographic systems have historically been designed under the assumption that adversaries are limited to classical computational models, an assumption that underpins the security of widely deployed public key algorithms based on the hardness of problems such as integer factorization and discrete logarithms [4].

The emergence of quantum computing challenges this assumption by introducing new computational paradigms capable of altering established security boundaries. In particular, quantum algorithms threaten to undermine the mathematical foundations of widely used public key systems [5]. As a response to this challenge, the concept of post-quantum cryptography has emerged, aiming to develop cryptographic algorithms that remain secure against both classical and quantum attacks [6]. Rather than relying on factorization or discrete logarithm problems, post-quantum algorithms are based on alternative mathematical structures that are currently believed to remain intractable even for quantum computers [7].

Recent standardization efforts have led to the selection of ML-KEM as a post-quantum key encapsulation mechanism suitable for widespread adoption [2]. Although the theoretical security foundations of ML-KEM have been extensively analyzed, much of the existing research focuses primarily on cryptographic proofs, protocol-level integrations such as TLS, or performance-optimized implementations in low-level programming languages [8]. Comparatively, limited attention has been given to evaluating the practical readiness of post-quantum cryptographic algorithms when deployed within modern high-level software stacks operating on classical computing platforms.

Despite significant progress in formal security analysis, theoretical guarantees alone do not fully capture the practical behavior of cryptographic algorithms. Real-world deployments introduce additional factors such as execution environments, software frameworks, resource constraints, and system interactions, all of which can influence performance and usability [9]. Consequently, evaluating cryptographic readiness requires empirical investigation beyond abstract security models.

The relevance of classical computing platforms in this context is driven by their dominance in existing and near-term infrastructure. Most applications that will adopt post-quantum cryptography will continue to operate on classical hardware for the foreseeable future [10]. Ensuring that post-quantum algorithms can be executed efficiently and reliably

on such platforms is therefore essential for realistic adoption strategies.

This paper addresses this gap by presenting an end-to-end implementation of the ML-KEM key encapsulation mechanism within a modern full-stack architecture based on a .NET 9 backend API and an Angular frontend application. The proposed system is executed entirely on classical hardware and is designed to measure critical performance metrics, including key generation latency, encapsulation and decapsulation time, and overall request-response behavior across application layers. In addition, the frontend component enables visualization and interaction with cryptographic operations, supporting a more transparent demonstration of post-quantum security concepts.

The main contribution of this work is an empirical evaluation of post-quantum cryptographic readiness on classical systems, demonstrating that ML-KEM¹ can be practically deployed in real-world software environments without reliance on specialized hardware. The results provide empirical evidence that post-quantum cryptography is not only theoretically secure but also feasible and efficient for contemporary applications, thereby supporting its adoption in future secure systems.

The implementation of the proposed system², including both backend and frontend components, is publicly available for reproducibility and further research via an online repository and live demonstration.

This paper is organized as follows: Section II reviews related work, Section III presents the system architecture and implementation, and Section IV discusses performance evaluation, results, and security analysis. Finally, the paper concludes by summarizing the key findings and insights derived from all parts of the study.

II. RELATED WORK

Post-quantum migration has accelerated following the final standardization of post-quantum cryptographic algorithms by the National Institute of Standards and Technology (NIST). In particular, ML-KEM (formerly CRYSTALS-Kyber) was selected as the primary post-quantum key encapsulation mechanism and standardized as FIPS 203, defining three parameter sets (ML-KEM-512, ML-KEM-768, and ML-KEM-1024) intended for general-purpose key establishment [11]. The standard specifies algorithmic structure, parameter selection, and security guarantees but intentionally abstracts away from application-level deployment and system integration considerations.

A substantial body of prior work evaluates post-quantum cryptographic performance through low-level implementations and microbenchmarks, typically focusing on CPU cycle counts, execution time distributions, memory footprint, and parameter trade-offs [8], [12]. The Open Quantum Safe (OQS) project is an open-source initiative that provides a collection of reference and optimized implementations of post-quantum cryptographic algorithms,

together with benchmarking and integration tools for real-world experimentation [13]. While these studies provide essential insight into raw algorithmic cost, they operate at a low abstraction level and do not capture overhead introduced by managed runtimes, application frameworks, serialization layers, or client-server communication.

In parallel, protocol-level research and standardization efforts have focused on integrating post-quantum and hybrid key exchange mechanisms into widely deployed secure communication protocols. The Internet Engineering Task Force (IETF) has proposed hybrid key exchange constructions for TLS 1.3, combining classical elliptic-curve Diffie-Hellman (ECDHE) with post-quantum mechanisms such as ML-KEM to preserve security even if one cryptographic assumption is later compromised [14]. These designs introduce hybrid groups such as X25519MLKEM768 and analyze handshake structure, message sizes, and compatibility with existing TLS infrastructure.

Large-scale experimental deployments further demonstrate the feasibility of post-quantum cryptography in real-world environments. Cloudflare has reported multiple post-quantum and hybrid TLS experiments measuring handshake latency, payload expansion, and operational constraints at Internet scale [15]. Similar measurement-oriented studies have analyzed post-quantum handshake performance in TLS and QUIC across different network conditions, focusing on latency and bandwidth overhead [16]. These works provide valuable protocol- and network-level insights but typically treat applications as black boxes and do not analyze internal backend execution or full-stack request lifecycles.

Unlike classical public-key algorithms such as RSA and elliptic curve cryptography, which depend on integer factorization or discrete logarithm problems vulnerable to Shor's quantum algorithm [1], ML-KEM relies on lattice-based hardness assumptions that remain resistant to known quantum algorithms [12]. Although Grover's algorithm can provide a quadratic speedup for brute-force search [3], its impact is mitigated through appropriate parameter selection defined in the NIST standardization process [11].

Importantly, the quantum resistance of ML-KEM is independent of the execution platform, as its security is derived from the hardness of the underlying Module-LWE problem rather than hardware-specific assumptions [2], [6]. The algorithm maintains its security properties when executed on classical computing hardware, as its resistance derives from mathematical hardness rather than specialized hardware requirements [5]. Consequently, deploying ML-KEM on classical systems enables quantum-ready security without requiring access to quantum computing resources.

However, despite extensive coverage at the algorithmic, systems, and protocol layers, fewer studies investigate post-quantum readiness as an end-to-end full-stack application

¹ <https://github.com/ErantKurtaliqi/ML-KEM/>

² <https://ml-kem.onrender.com/>

behavior using modern managed runtimes and web application stacks. In particular, the interaction between backend cryptographic execution, framework-level overhead, serialization, and frontend-backend communication remains underexplored. This paper complements existing literature by positioning ML-KEM within a .NET 9 backend and Angular frontend and by quantifying both backend cryptographic cost and end-to-end latency with explicit overhead decomposition.

Table 1 summarizes the positioning of this work relative to prior literature across different levels of abstraction and evaluation.

TABLE 1 - POSITIONING OF THIS WORK RELATIVE TO PRIOR LITERATURE

<i>Study / Stack</i>	<i>Environment</i>	<i>What is Measured</i>	<i>Notes vs. This Work</i>
Low-level implementations / OQS benchmarks [12], [13]	C / systems-level	Microbenchmarks (CPU cycles, execution time distributions, memory usage)	Algorithm-focused; no application or full-stack context
TLS 1.3 hybrid design & ECDHE+ML-KEM groups (IETF) [14]	Protocol-level	Handshake construction, key exchange groups, message sizes	Protocol integration; abstracts application-layer behavior
Large-scale TLS PQ experiments (Cloudflare) [15], [17]	Real-world deployment	Feasibility, handshake latency, payload overhead	TLS-focused; not managed runtime or frontend-backend lifecycle
Post-quantum Internet adoption measurements [4]	Production traffic	Adoption metrics, deployment constraints	Internet-scale view; not application-level profiling
Comparative analyses of PQ TLS/QUIC handshakes	TLS/QUIC studies	Handshake latency and bandwidth overhead	Network-centric evaluation; limited insight into application internals
Proposed model	Full-stack (.NET 9 + Angular)	Backend KEM execution + end-to-end latency + overhead decomposition	Bridges algorithm cost to real application behavior in managed runtimes

III. SYSTEM ARCHITECTURE AND IMPLEMENTATION

The proposed system is designed as a full-stack application consisting of a backend service and a frontend client. The architecture follows a service-oriented design in which cryptographic operations are encapsulated within dedicated backend APIs, while the frontend application provides user interaction and result visualization. Service-oriented architectures are widely used in modern distributed systems to isolate critical functionality and improve system modularity [10].

The backend is implemented using .NET 9 and exposes endpoints responsible for key generation, encapsulation, and decapsulation operations. Centralizing cryptographic logic

within the backend ensures controlled execution of security-critical operations and prevents exposure of secret key material to the client environment. The frontend, implemented using Angular, communicates with the backend through HTTPS requests and serves primarily as a visualization and interaction layer for initiating cryptographic operations and observing. This architecture reflects common patterns used in modern web-based applications and provides a realistic environment for evaluating the deployment of post-quantum cryptography on classical computing platforms.

A. Threat Model

The security evaluation is conducted under a threat model that reflects realistic deployment conditions for modern web-based systems. The adversary is assumed to have full access to the public communication channel between the frontend and backend components and may observe, intercept, modify, or replay network traffic. This corresponds to the standard Dolev–Yao network attacker model [18].

The system is evaluated against both classical and quantum-capable adversaries. In the quantum threat scenario, the adversary is assumed to possess access to a sufficiently powerful quantum computer capable of executing algorithms such as Shor’s algorithm for integer factorization and discrete logarithms [1] and Grover’s algorithm for unstructured search [3].

However, the adversary is not assumed to have access to secret key material stored in backend memory nor the ability to compromise the operating system or runtime environment.

Within this model, the following attack scenarios are considered:

- Passive eavesdropping on communication channels
- Active manipulation of transmitted ciphertexts
- Replay attacks targeting encapsulated key material
- Classical and quantum cryptanalysis of public key material

The security of the system against these attacks relies on the post-quantum security guarantees of ML-KEM, whose hardness is based on the Module Learning with Errors (Module-LWE) problem [12], [19].

Side-channel attacks, fault injection, operating system compromise, and attacks targeting the frontend execution environment are considered outside the scope of this study.

B. System Assumptions

The evaluation assumes that the backend service operates within a trusted execution environment and that the .NET runtime provides correct implementations of cryptographic primitives. Secure random number generation is assumed to be available through the operating system. Modern operating systems provide cryptographically secure random number generators designed for security-sensitive applications [20].

The frontend application is treated as an untrusted environment and does not store long-term secret key material. All sensitive cryptographic operations are executed exclusively within the backend service.

The primary objective of this work is to demonstrate that post-quantum key encapsulation using ML-KEM can be securely and efficiently integrated into a modern full-stack

application architecture operating entirely on classical hardware.

Secure random number generation is provided through the .NET RandomNumberGenerator class, which relies on the underlying operating system's cryptographically secure random number generator (CSPRNG), such as CryptGenRandom on Windows or /dev/urandom on Unix-based systems.

C. ML-KEM Backend Implementation

ML-KEM is a lattice-based key encapsulation mechanism designed to provide secure key exchange in a post-quantum context. Lattice-based cryptography relies on the hardness of mathematical problems defined over high-dimensional lattices, such as the Shortest Vector Problem (SVP) and the Learning With Errors (LWE) problem, which are believed to remain computationally intractable even for quantum computers. Its security relies on the hardness of the Module Learning With Errors (Module-LWE) problem [12], [19], which is currently believed to be resistant to both classical and quantum attacks.

In the proposed system, ML-KEM is implemented within the .NET 9 backend as a dedicated cryptographic service responsible for key generation, encapsulation, and decapsulation. The implementation uses the Bouncy Castle cryptographic library for ML-KEM operations, specifically through the MLKemKeyPairGenerator, MLKemEncapsulator, and MLKemDecapsulator classes. Since native post-quantum cryptographic primitives are not currently provided through the standard .NET System.Security.Cryptography namespace, the use of Bouncy Castle enables a practical and reproducible integration of ML-KEM into a modern managed runtime environment.

Cryptographic operations are exposed through backend API endpoints responsible for key generation, encapsulation, and decapsulation. This design allows cryptographic operations to be invoked systematically and evaluated as part of the overall system behavior rather than as isolated benchmarks.

D. Frontend Integration and System Interaction

The frontend component provides a user interface for interacting with the cryptographic backend and visualizing execution results. Implemented using Angular, the frontend initiates backend requests, displays cryptographic outputs, and presents timing information associated with each operation.

Through this interactive interface, users can trigger key generation, encapsulation, and decapsulation operations without requiring detailed knowledge of the underlying cryptographic processes. This integration enables visualization of execution flow and performance behavior across application layers.

By incorporating a frontend layer, the system extends beyond traditional cryptographic benchmarks and provides a demonstrative platform suitable for experimentation, evaluation, and educational purposes.

E. Experimental Environment

All experiments were conducted on a classical computing platform representative of a modern development environment. Both backend and frontend components were executed on the same physical machine in order to minimize external network variability while preserving realistic application-layer communication behavior. The detailed configuration of the experimental environment is presented in Table 2.

TABLE 2 - EXPERIMENTAL ENVIRONMENT CONFIGURATION

Component	Specification
Processor	13th Gen Intel(R) Core(TM) i5-13400
Base Clock Frequency	2.50 GHz
RAM	16.0 GB
Operating System	Windows 11 Pro (64-bit)
Backend Framework	.NET 9 (Release build)
Runtime Environment	.NET CLR
Frontend Framework	Angular 19
Execution Mode	Release
Backend Protocol	HTTPS
Frontend Protocol	HTTPS
Deployment	Localhost (client and server on same machine)
Network Interface	Loopback (127.0.0.1)

The backend service was deployed using the .NET 9 runtime in Release mode and exposed API endpoints over HTTPS. The Angular frontend was executed as a production build and communicated with the backend through the local loopback interface. This configuration preserves realistic application-layer communication while minimizing network variability. A reproducibility checklist and experimental setup details are summarized in Table 3.

TABLE 3 - EXPERIMENTAL SETUP & REPRODUCIBILITY CHECKLIST

Hardware	Intel i5-13400, 16GB RAM
OS	Windows 11 Pro 64-bit
Backend	.NET 9 Release, Stopwatch instrumentation
Frontend	Angular 19 production build
Browser	Chrome 121
Runs	Backend N=1000 per variant; End-to-end N=500 per variant
Warm-up	A warm-up phase of 100 iterations was performed prior to data collection to allow the .NET CLR to complete just-in-time (JIT) compilation and stabilize runtime behavior.
Timing source	Backend: server-side timestamps (Stopwatch); End-to-end: client-side request lifecycle timing
Network	Localhost loopback (127.0.0.1); HTTPS backend
Outliers	Attributed to GC and thread scheduling; reported via max and std dev

F. Measurement Methodology

Backend cryptographic performance was measured within the .NET 9 service using the built-in Stopwatch class. Timing instrumentation was placed immediately before and after ML-KEM encapsulation and decapsulation calls to measure pure cryptographic execution time while excluding network and application overhead, following common benchmarking practices for cryptographic performance evaluation.

End-to-end latency was measured at the application level by capturing the time between request initiation in the Angular frontend and response reception. These measurements include frontend processing, protocol handling, backend execution, and response serialization, providing a realistic representation of full-stack behavior.

To ensure stable measurements, a warm-up phase was performed before data collection. Backend measurements were conducted over 1000 iterations per ML-KEM variant, while 500 frontend-initiated requests were used for end-to-end measurements. Mean, minimum, maximum, and standard deviation values were recorded to capture both central tendency and variability of the measurements. Occasional outliers are attributed to runtime effects such as garbage collection and thread scheduling.

All measurements were conducted in a controlled environment with minimized external interference to ensure reproducibility and consistency across experimental runs.

G. Workload Strategy

While this study focuses on sequential execution to isolate baseline performance characteristics, it does not evaluate concurrent workloads or high-throughput scenarios. Therefore, the results should be interpreted as baseline feasibility rather than full production scalability. Future work will extend the evaluation to include concurrent request handling and stress testing.

Backend measurements were obtained through repeated encapsulation and decapsulation operations within a controlled execution loop. End-to-end measurements were collected by issuing discrete HTTPS requests from the frontend to the backend API. No artificial load generation or stress-testing mechanisms were introduced.

This workload configuration is intentionally selected to evaluate the baseline performance and practical feasibility of integrating ML-KEM into a modern full-stack application architecture rather than measuring high-concurrency throughput.

IV. PERFORMANCE EVALUATION, RESULTS AND SECURITY ANALYSIS

The performance evaluation analyzes key generation latency, encapsulation and decapsulation time, and overall request-response behavior within the proposed full-stack architecture. The results indicate that ML-KEM operations can be executed efficiently on classical hardware without introducing significant latency.

End-to-end measurements show that the additional overhead introduced by frontend-backend communication remains within acceptable bounds, confirming that post-

quantum cryptography can be integrated into modern applications without prohibitive performance costs. In addition to standard deviation, confidence intervals (95%) can be derived assuming an approximately normal distribution of measurements. Given the large number of iterations (N=1000 for backend and N=500 for end-to-end measurements), the reported mean values provide a stable estimate of expected performance.

TABLE 4 - END-TO-END EXECUTION TIME OF ML-KEM VARIANTS OVER 500 REQUESTS

<i>ML-KEM Variant</i>	<i>Payload Size (kB)</i>	<i>Mean Time (ms)</i>	<i>Min (ms)</i>	<i>Max (ms)</i>	<i>Std Dev (ms)</i>
ML-KEM-512	3.4	2.39	1.57	12.97	0.84
ML-KEM-768	4.89	2.84	1.76	16.43	1.02
ML-KEM-1024	6.43	2.92	1.76	43.25	1.24

Table 4 presents the end-to-end execution time of ML-KEM variants measured over 500 HTTPS requests issued from the Angular frontend to the .NET 9 backend API. The measured latency includes frontend processing, HTTPS communication, backend routing and middleware execution, serialization, and cryptographic computation.

All ML-KEM variants achieve mean end-to-end latencies below 3ms, demonstrating that post-quantum key encapsulation can be integrated into modern web applications without noticeable response delays.

TABLE 5 - BACKEND EXECUTION TIME OF ML-KEM ENCAPSULATION OVER 1000 ITERATIONS

<i>ML-KEM Variant</i>	<i>Mean Time (ms)</i>	<i>Min (ms)</i>	<i>Max (ms)</i>	<i>Std Dev (ms)</i>
ML-KEM-512	0.0649	0.0406	2.4258	0.0217
ML-KEM-768	0.0723	0.0605	1.1364	0.0189
ML-KEM-1024	0.0789	0.0697	2.0904	0.0246

Table 5 reports backend-only execution time of the ML-KEM encapsulation operation measured over 1000 iterations using server-side Stopwatch instrumentation. The results isolate pure cryptographic computation, excluding network and application-layer overhead.

Encapsulation operations execute in approximately 0.065–0.079ms, confirming that the computational overhead of ML-KEM on classical hardware is minimal.

TABLE 6 - BACKEND EXECUTION TIME OF ML-KEM DECAPSULATION OVER 1000 ITERATIONS

<i>ML-KEM Variant</i>	<i>Mean Time (ms)</i>	<i>Min (ms)</i>	<i>Max (ms)</i>	<i>Std Dev (ms)</i>
ML-KEM-512	0.0485	0.0281	0.9643	0.0172
ML-KEM-768	0.1364	0.1183	2.1401	0.0315

<i>ML-KEM Variant</i>	<i>Mean Time (ms)</i>	<i>Min (ms)</i>	<i>Max (ms)</i>	<i>Std Dev (ms)</i>
ML-KEM-1024	0.1842	0.1254	12.0408	0.0679

Table 6 presents the backend execution time of ML-KEM decapsulation across different parameter sets. Decapsulation measurements also remain within the sub-millisecond range across all security parameter sets. Slightly higher execution times for larger parameter sets are expected due to increased cryptographic complexity.

TABLE 7 - BACKEND EXECUTION TIME VS END-TO-END LATENCY AND COMMUNICATION OVERHEAD

<i>ML-KEM Variant</i>	<i>Backend Mean (ms)</i>	<i>End-to-End Mean (ms)</i>	<i>Overhead (ms)</i>	<i>Overhead (%)</i>
ML-KEM-512	0.0649	2.39	2.3251	97.29
ML-KEM-768	0.0723	2.84	2.7677	97.46
ML-KEM-1024	0.0789	2.92	2.8411	97.30

Table 7 compares backend execution time with end-to-end latency and quantifies communication overhead. The comparison between backend execution time and full request latency shows that the cryptographic computation accounts for less than 3% of total latency, while over 97% originates from application-layer processing and frontend-backend communication. These results demonstrate that ML-KEM does not represent a performance bottleneck in full-stack architectures.

A. Baseline Comparison with Classical Key Exchange

TABLE 8 - BASELINE COMPARISON BETWEEN POST-QUANTUM AND CLASSICAL KEY EXCHANGE

<i>Algorithm</i>	<i>Security Level</i>	<i>Mean Execution Time (ms)</i>	<i>Payload Size (kB)</i>
RSA-2048	Classical	~0.20	~0.27
ECDH P-256	Classical	~0.05	~0.09
ML-KEM-512	Post-Quantum	0.0649	3.4
ML-KEM-768	Post-Quantum	0.0723	4.89
ML-KEM-1024	Post-Quantum	0.0789	6.43

Table 8 provides a baseline comparison between post-quantum and classical key exchange mechanisms. The reported RSA and ECDH performance values are derived from external literature and may reflect different hardware configurations and implementation environments. As such, this comparison should be interpreted as indicative rather than strictly equivalent.

Although ML-KEM introduces larger payload sizes and slightly higher execution times compared to classical key exchange mechanisms, the observed overhead remains within

practical limits. A fully controlled comparison would require evaluating all algorithms within the same experimental setup, which is left for future work.

TABLE 9 - REMOTE END-TO-END LATENCY OF ML-KEM VARIANTS

<i>ML-KEM Variant</i>	<i>Mean (ms)</i>	<i>Min (ms)</i>	<i>Max (ms)</i>	<i>Std. Dev (ms)</i>
ML-KEM-512 (Remote)	66.95	46.21	240.96	20.31
ML-KEM-768 (Remote)	69.64	49.96	330.35	23.37
ML-KEM-1024 (Remote)	70.95	53.21	340.96	26.22

Table 9 presents the remote end-to-end latency measurements for ML-KEM variants. When deployed in a remote Internet-based environment, mean end-to-end latency ranges between 67ms and 71ms. In such deployments, network transmission and cloud infrastructure effects dominate total response time, meaning that differences in cryptographic computation and payload size have minimal influence on application-level latency.

TABLE 10 - BACKEND EXECUTION TIME OF ML-KEM KEY GENERATION OVER 1000 ITERATIONS

<i>ML-KEM Variant</i>	<i>Mean (ms)</i>	<i>Min (ms)</i>	<i>Max (ms)</i>	<i>Std Dev (ms)</i>
ML-KEM-512	0.1124	0.0851	1.8427	0.0632
ML-KEM-768	0.1347	0.1023	2.1945	0.0718
ML-KEM-1024	0.1589	0.1214	3.0672	0.0896

Table 10 shows the backend execution time of ML-KEM key generation over 1000 iterations. All variants operate within sub-millisecond range, with slightly higher times for larger parameter sets due to increased complexity. The low standard deviation indicates stable performance, while occasional higher values are attributed to runtime effects such as garbage collection. Overall, key generation introduces minimal overhead in the system.

B. Discussion

Overall, the results demonstrate that the dominant contributor to system latency is application-layer processing and communication overhead, rather than the cryptographic computation itself. Backend measurements confirm that ML-KEM operations execute in sub-millisecond time, while end-to-end request latency remains well within acceptable limits for modern web applications.

These findings indicate that the computational cost of ML-KEM is negligible relative to overall system latency. As a result, performance bottlenecks in full-stack applications are primarily driven by framework-level overhead, network communication, and request processing rather than cryptographic operations.

C. Security and Quantum Readiness Analysis

Quantum readiness in practice involves the ability to deploy cryptographic mechanisms that are resilient to anticipated quantum attacks while remaining compatible with existing systems. The proposed implementation demonstrates that ML-KEM satisfies these requirements when executed on classical platforms [11], [12].

By integrating cryptographic operations into a complete application workflow, this study provides evidence of secure and functional deployment in a realistic system setting. The use of ML-KEM ensures that key exchange operations are based on quantum-resistant assumptions, specifically the hardness of the Module-LWE problem, while maintaining compatibility with existing application architectures.

In practical deployments, post-quantum migration is often implemented through hybrid key exchange mechanisms that combine classical and post-quantum algorithms. For example, ongoing IETF standardization efforts recommend hybrid constructions such as X25519 combined with ML-KEM to ensure security against both classical and quantum adversaries during the transition period. While this work evaluates a pure ML-KEM-based approach, the proposed architecture is compatible with hybrid integration strategies and can be extended accordingly.

1) Secure Session Construction

ML-KEM provides a shared secret but does not encrypt application data by itself. In our system, the encapsulated shared secret is immediately passed through a key derivation function (KDF) to derive symmetric session keys. These session keys are used with an authenticated encryption scheme (AEAD) to protect confidentiality and integrity [21] of application messages. Session keys are short-lived and scoped to a single session to limit exposure.

2) Key Handling

Private keys and derived session keys are confined to the backend. Secret material is stored only in process memory and is not persisted. After use, secret buffers are cleared when feasible to reduce lifetime in memory.

3) Replay Protection

The system includes replay resistance by binding session establishment to a unique request identifier and rejecting repeated ciphertext/session identifiers observed within the session window.

4) Transport Security

TLS is treated as orthogonal; experiments isolate KEM computation and application-layer overhead, but the architecture supports deployment over TLS in real settings [14].

V. CONCLUSION AND FUTURE WORK

This paper presented an end-to-end implementation of the ML-KEM post-quantum key encapsulation mechanism within a modern full-stack architecture executed entirely on classical hardware. The results demonstrate that post-quantum cryptography can be practically deployed using contemporary software frameworks while maintaining acceptable performance characteristics.

By combining empirical measurement with real-world application design, this work provides evidence that quantum-resistant cryptographic mechanisms are not only theoretically secure but also operationally feasible in practical software environments. The evaluation confirms that ML-KEM can be integrated into modern application architectures without introducing significant performance

overhead, supporting the broader transition toward post-quantum security in classical computing systems.

Despite these promising results, this study has several limitations that should be considered when interpreting the findings. First, the evaluation was conducted primarily in a controlled single-machine environment using localhost deployment to isolate cryptographic and application-layer performance. Although additional measurements were performed in a hosted web environment, these experiments remain limited in scale and do not fully capture realistic distributed system conditions, such as network congestion, multi-user load, or cloud infrastructure variability. Second, the workload model is strictly sequential and does not reflect concurrent request handling or high-throughput scenarios typical of production systems. Third, the experiments were conducted on a single hardware configuration, which limits the generalizability of the results across different platforms and deployment environments. Finally, the analysis focuses primarily on ML-KEM and does not include a broader comparison with other post-quantum or hybrid key exchange mechanisms within the same experimental framework.

Future work will extend this evaluation by incorporating newer runtime environments, such as .NET 10, to assess potential performance improvements in managed execution. In addition, larger experimental sample sizes (e.g., $N > 10,000$) will be used to improve statistical robustness and reduce measurement variance. Further studies will include concurrent workload scenarios, integration with cloud-based and distributed systems, and evaluation across different hardware architectures, including ARM-based environments. These extensions will provide a more comprehensive understanding of post-quantum cryptographic performance and support the development of scalable and secure systems in preparation for the post-quantum era.

REFERENCES

- [1] P. W. Shor, "Algorithms for Quantum Computation: Discrete Logarithms and Factoring," in *Proceedings 35th Annual Symposium on Foundations of Computer Science*, Santa Fe, NM, USA, 1994.
- [2] (NIST), National Institute of Standards and Technology, "Module-Lattice-Based Key-Encapsulation Mechanism Standard," Federal Information Processing Standards Publication 203, National Institute of Standards and Technology, U.S. Department of Commerce, 2024.
- [3] M. Mosca, "Cybersecurity in an Era with Quantum Computers: Will We Be Ready?," *IEEE Security & Privacy*, DOI: <https://doi.org/10.1109/MSP.2018.3761723>, vol. 16, no. 5, pp. 38 - 41, 2018.
- [4] Alkim, Erdem et al., "CRYSTALS-Kyber: Algorithm Specifications and Supporting Documentation," International Association for Cryptologic Research, 2021.
- [5] Daniel J. Bernstein, Johannes Buchmann, Erik Dahmen, Post-Quantum Cryptography, Berlin, Heidelberg: Springer Berlin, Heidelberg, 2009, DOI: <https://doi.org/10.1007/978-3-540-88702-7>.
- [6] Lily Chen et al., "Report on Post-Quantum Cryptography (NISTIR 8105)," NISTIR, 04 2016. [Online]. Available: <https://nvlpubs.nist.gov/nistpubs/ir/2016/NIST.IR.8105.pdf>. [Accessed 30 December 2025].
- [7] Jonathan Katz, Yehuda Lindell, Introduction to Modern Cryptography, Boca Raton: CRC Press, 2020.
- [8] O. Regev, "On Lattices, Learning with Errors, Random Linear Codes, and Cryptography," *Journal of the ACM (JACM)*, DOI: <https://doi.org/10.1145/1568318.1568324>, vol. 56, no. 6, pp. 1-40, 2009.

- [9] Vadim Lyubashevsky, Chris Peikert, Oded Regev, "On Ideal Lattices and Learning with Errors over Rings," *Advances in Cryptology – EUROCRYPT 2010*, vol. 6110, no. Lecture Notes in Computer Science (LNCS), pp. 1-23, 2010.
- [10] T. Erl, "Service-Oriented Architecture: Concepts, Technology, and Design," Prentice Hall PTR, Upper Saddle River, 2005.
- [11] Jeffrey Hoffstein, Jill Pipher, Joseph H. Silverman, An Introduction to Mathematical Cryptography, 2008, DOI: <https://doi.org/10.1007/978-0-387-77993-5>; Springer New York, NY, New York.
- [12] C. Peikert, A Decade of Lattice Cryptography, Boston: Now Publishers Inc., 2016.
- [13] J. Bos et al., "CRYSTALS - Kyber: A CCA-Secure Module-Lattice-Based KEM," in *2018 IEEE European Symposium on Security and Privacy (EuroS&P)*, doi: [10.1109/EuroSP.2018.00032](https://doi.org/10.1109/EuroSP.2018.00032), London, UK, 2018.
- [14] Konstantina Souvatzidaki, and Konstantinos Limniotis, "Hybrid Post-Quantum Key Exchange in TLS 1.3," *Cryptography*, DOI: <https://doi.org/10.3390/cryptography9040073>, vol. 9, no. 4, pp. 1-24, 2025.
- [15] Bas Westerbaan, Cefan Daniel Rubin, "Defending against future threats: Cloudflare goes post-quantum," Cloudflare, Inc., 03 10 2022. [Online]. Available: <https://blog.cloudflare.com/post-quantum-for-all/>. [Accessed 3 January 2026].
- [16] Peter Schwabe, Douglas Stebila and Thom Wiggers, "Post-Quantum TLS Without Handshake Signatures," in *ACM CCS / ose versionet e IETF drafts / Cloudflare experiments*; <https://dl.acm.org/doi/10.1145/3372297.3423350>, New York, NY, United States, 2020.
- [17] Krzysztof Kwiatkowski, Nick Sullivan, Adam Langley, Dave Levin, Alan Mislove, Cloudflare, Google, University of Maryland and Northeastern University, "Measuring TLS key exchange with post-quantum KEM," in *Second NIST Post-Quantum Cryptography Standardization Conference*, Santa Barbara, California, USA, 2019.
- [18] D. Dolev and A. Yao, "On the security of public key protocols," *IEEE Transactions on Information Theory*, doi: [10.1109/TIT.1983.1056650](https://doi.org/10.1109/TIT.1983.1056650), vol. 29, no. 2, p. 198–208, 1983.
- [19] Lyubashevsky, Vadim; Peikert, Chris; Regev, Oded, "On Ideal Lattices and Learning With Errors Over Rings," *Journal of the ACM (JACM)*, DOI: <https://doi.org/10.1145/2535925>, vol. 60, no. 6, pp. 1 - 35, 2013.
- [20] Barker, E. and Kelsey, J., "Recommendation for Random Number Generation Using Deterministic Random Bit Generators," National Institute of Standards and Technology, DOI: <https://doi.org/10.6028/NIST.SP.800-90Ar1>, Gaithersburg, 2015.
- [21] P. Rogaway, "Authenticated Encryption: Theory and Practice," in *Authenticated encryption. In: van Tilborg, H.C.A. (eds) Encyclopedia of Cryptography and Security*, Davis, Springer, Boston, MA, 2002, DOI: https://doi.org/10.1007/0-387-23483-7_15.